SOFTWARE DESIGN DOCUMENT (v2)

Table of Contents

SOFTWARE DESIGN DOCUMENT (v2) 1
Document Authorship 2
1. System Overview 2
2. System Context 2
3. Key Features and Functionality 2
4. Assumptions and Dependencies 2
5. Architectural Design 3
5.1 High-Level Architecture 3
5.2 Architectural patterns and styles 3
5.3 Rationale for architectural decisions 3
5.4 Diagram placement: Component Diagram 3
6. Component Design 3
6.1 Subsystems and modules 3
6.2 Responsibilities 4
6.3 Interfaces between components 4
6.4 Diagram placement: Class Diagram 4
7. Data Design 5
7.1 Data model 5
7.2 Data storage 5
7.3 Data flow 5
7.4 Data validation rules 5
8. Design Patterns 5
8.1 Applied Pattern A: Builder (implemented) 5
8.2 Pattern B context: Strategy-oriented classification flow (current rule-based implementation) 6
9. Implementation Notes 6
10. User Interface Design 6
11. External Interfaces 6
12. Performance Considerations 6
13. Error Handling and Logging 7
14. Design for Testability 7

15. Deployment and Installation Design 7
15.1 Deployment model 7
15.2 Diagram placement: Deployment Diagram 7
16. Sequence and Runtime Flow 8
17. Product Backlog to Sprint Backlog Transition (Current Phase) 8
18. Open Issues and Future Work 8
19. Document-Specific Task Matrix 8

Document Authorship

- Contributor 1: Kerem Elma
- Contributor 2: Mehmet Eski

1. System Overview

LifeSync is a health-oriented software system that supports user registration, authentication, profile management, and personalized recommendation generation. The web application handles account and profile operations, while the Python client collects extended user input and generates recommendations through an LLM provider (Ollama).

The system is currently implemented as a layered monolithic backend plus a separate client-side inference tool.

2. System Context

- Frontend: React application for login, dashboard, and profile editing.
- Backend: Node.js + Express REST API.
- Data: PostgreSQL user data storage.
- AI client: Python CLI for profile survey and recommendation generation.
- Ollama local API for text generation.

3. Key Features and Functionality

- User authentication (register, login, me, logout).
- Protected dashboard with BMI calculation.
- Editable profile (password, height, weight, age, gender).
- Partial profile updates through dynamic SQL generation.
- CLI-based recommendation generation from user survey data via Ollama.

4. Assumptions and Dependencies

- PostgreSQL is reachable and configured in backend environment variables.
- JWT secret is configured for production deployments.
- Ollama service is available at configured endpoint for CLI recommendation flow.

- Frontend and backend run in compatible local/network environment.

5. Architectural Design

5.1 High-Level Architecture

Chosen architecture style: Layered Architecture.

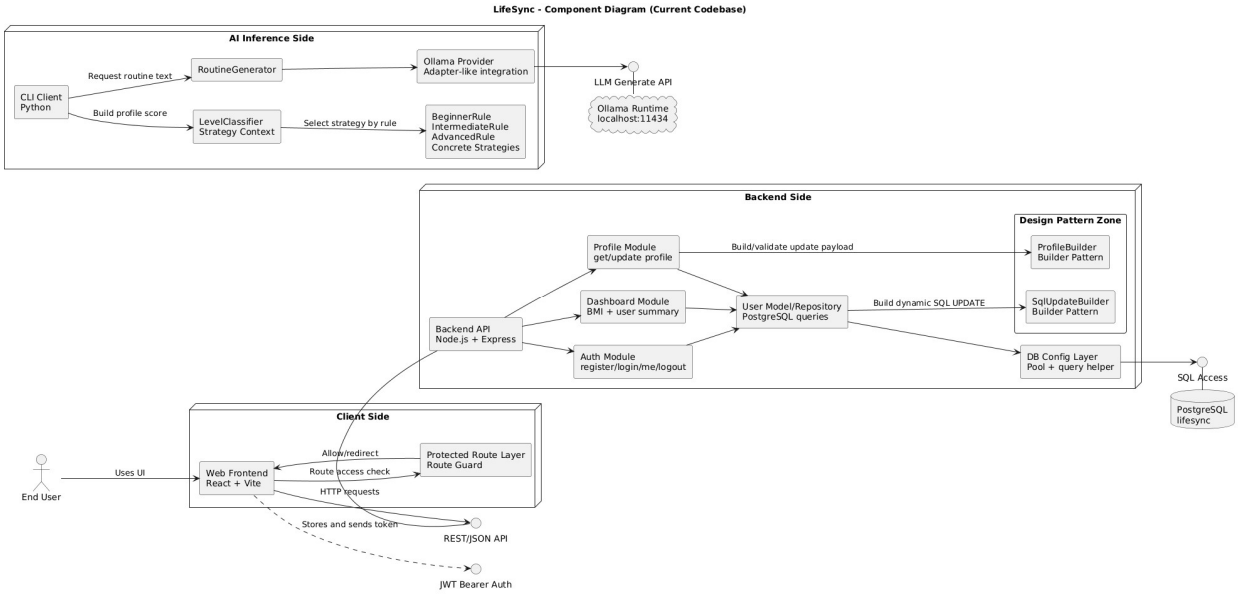
5.2 Architectural patterns and styles

- Layered backend separation: routes -> controllers -> models -> database.
- Builder pattern in profile update flow for validation and SQL statement construction.
- Rule-based classification in CLI recommendation flow (candidate for strict Strategy polymorphism extension).

5.3 Rationale for architectural decisions

- Layered design keeps responsibilities clear and supports maintainability.
- Builder-based profile update minimizes controller complexity and supports field extensibility.
- Separation between web operations and AI generation prevents coupling between critical API and LLM runtime behavior.

5.4 Diagram placement: Component Diagram



6. Component Design

6.1 Subsystems and modules

- Frontend module: AuthPage, Dashboard, EditProfile, route protection.
- Backend API module: Auth, Dashboard, Profile endpoints.
- Domain/data module: User model and DB config.
- Pattern module: ProfileBuilder, SqlUpdateBuilder.

- AI client module: survey collection, classification, prompt build, Ollama request.

6.2 Responsibilities

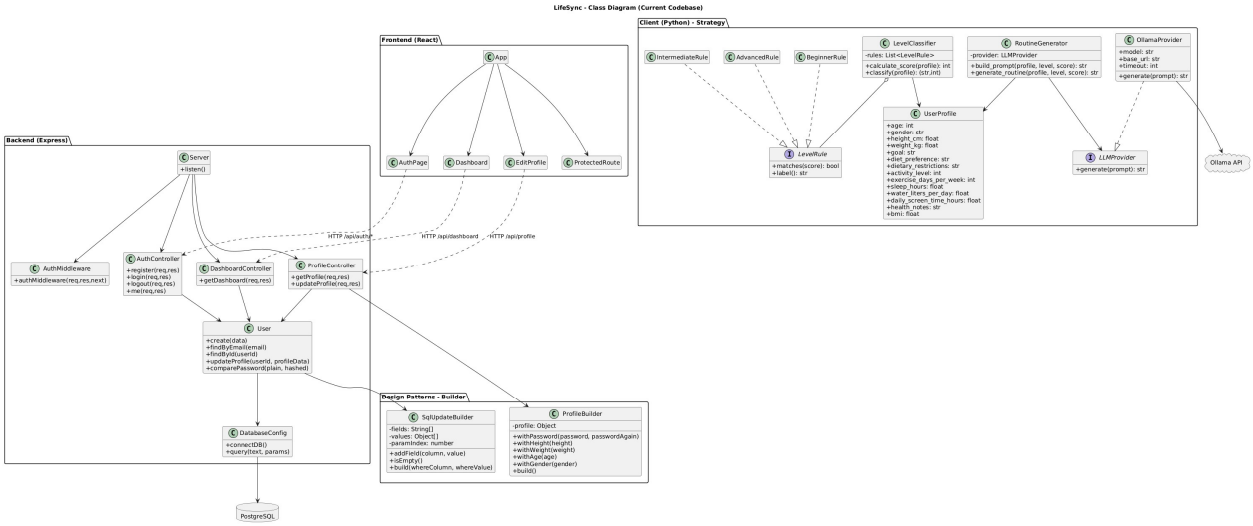
- AuthController: user onboarding and authentication token lifecycle.
- DashboardController: health summary generation (BMI).
- ProfileController: profile retrieval and update orchestration.
- ProfileBuilder: update payload validation/sanitization.
- SqlUpdateBuilder: dynamic SQL UPDATE statement generation.
- User model: database access and password hashing/comparison.

6.3 Interfaces between components

API interface examples

- POST /api/auth/register
  - Input: email, firstName, lastName, password, optional health fields
  - Output: token, user
- POST /api/auth/login
  - Input: email, password
  - Output: token, user
- GET /api/dashboard
  - Input: Bearer token
  - Output: user summary + BMI
- GET /api/profile
  - Input: Bearer token
  - Output: user profile
- PUT /api/profile
  - Input: Bearer token + updatable fields
  - Output: updated user profile

6.4 Diagram placement: Class Diagram



7. Data Design

7.1 Data model

Core entity: users

- user_id
- email
- first_name
- last_name
- password
- height
- weight
- age
- gender

7.2 Data storage

- Primary store: PostgreSQL.
- Access layer: pg pool via backend DB config module.

7.3 Data flow

- Frontend sends profile updates to backend.
- Backend validates and builds payload via builder.
- Model updates only provided fields.

7.4 Data validation rules

- Password minimum length check.
- Height, weight, age numeric bounds.
- Gender domain restriction (male/female).

8. Design Patterns

8.1 Applied Pattern A: Builder (implemented)

Context

Profile update accepts optional and partial fields.

Implementation points

- backend/src/builders/ProfileBuilder.js
- backend/src/builders/SqlUpdateBuilder.js
- backend/src/controllers/profileController.js
- backend/src/models/User.js

Intent and motivation

- Build update payload incrementally.
- Keep validations centralized.
- Generate dynamic SQL safely for partial updates.

Improvement impact

- Controller logic simplified.
- New profile field additions require fewer structural changes.
- Reduced risk of inconsistent field handling.

8.2 Pattern B context: Strategy-oriented classification flow (current rule-based implementation)

Context

User level is inferred from survey metrics before recommendation generation.

Current implementation points

- client/main.py (classify_user and scoring rules)

Note

Current implementation is rule-based and can be refactored into explicit Strategy classes without changing the surrounding workflow boundaries.

9. Implementation Notes

- Backend: Express, JWT auth middleware, PostgreSQL integration, bcrypt hashing.
- Frontend: React pages with protected routes.
- CLI: Python requests to Ollama endpoint for recommendation generation.

10. User Interface Design

- Auth page for login/register.
- Dashboard page for user summary and BMI.
- Edit Profile page for mutable fields.

11. External Interfaces

- REST APIs between frontend and backend.
- Ollama HTTP API for text generation in CLI flow.

12. Performance Considerations

- Database query paths are simple and indexed by key fields.
- Profile update updates only changed fields.
- CLI LLM generation is network/runtime dependent and potentially high-latency.

13. Error Handling and Logging

- Backend returns 4xx for validation/auth errors, 5xx for server failures.
- Controller-level try/catch with error responses.
- CLI returns connection/timeout diagnostics for Ollama failures.

14. Design for Testability

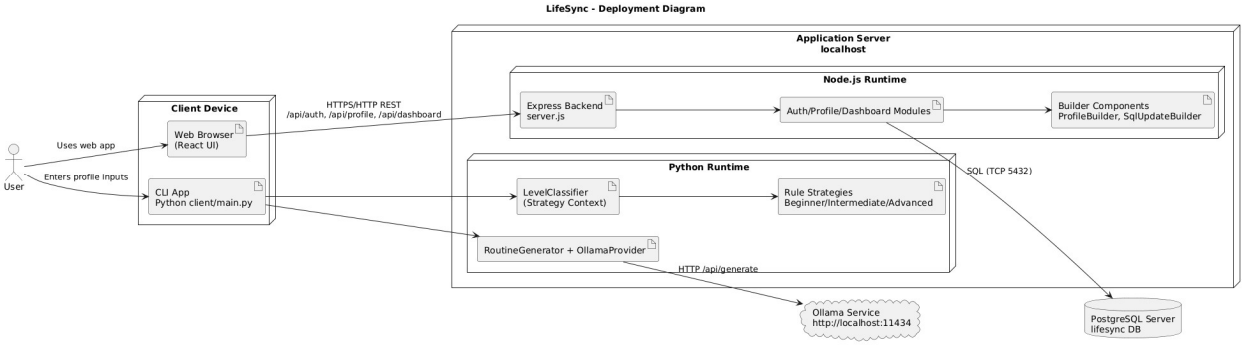
- Layered backend enables unit testing controllers/models separately.
- Builder classes are independently testable for validation behavior.
- API endpoints can be tested with token/no-token, valid/invalid payloads.

15. Deployment and Installation Design

15.1 Deployment model

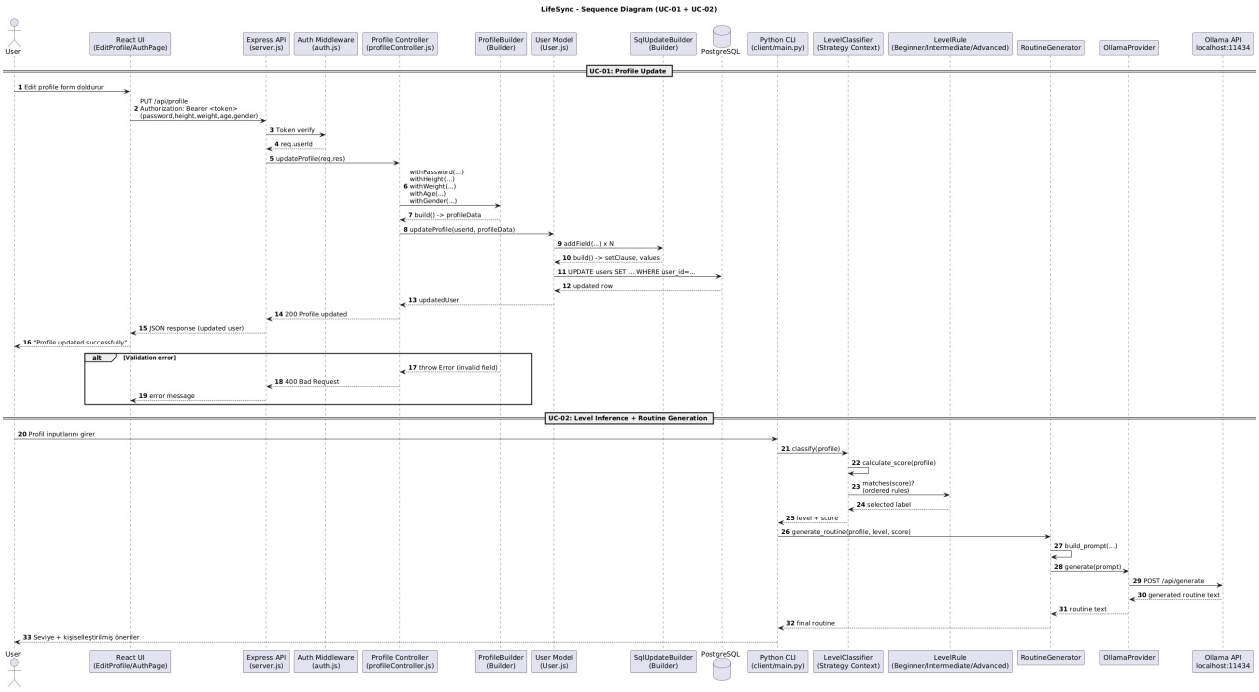
- Frontend app runtime.
- Backend API runtime.
- PostgreSQL server.
- Python CLI runtime.
- Ollama service.

15.2 Diagram placement: Deployment Diagram



16. Sequence and Runtime Flow

The sequence diagrams drawn for use cases UC-01 and UC-02 are shown below.



17. Product Backlog to Sprint Backlog Transition (Current Phase)

- High-level requirements mapped to UC-01 and UC-02.
- Sprint execution follows: Requirement -> Design -> Implementation -> Test.
- Product backlog remains living and is refined before each sprint.

18. Open Issues and Future Work

- Refactor CLI classification into explicit Strategy class hierarchy.
- Add automated test suite for builder classes and API contract tests.
- Add standardized error code taxonomy across endpoints.
- Consolidate document versioning and traceability matrix artifacts.

19. Document-Specific Task Matrix

Task	Owner	Output
Architecture update	Kerem Elma	Sections 1-5
Component and class documentation	Kerem Elma	Sections 6-8
Data and API documentation	Mehmet Eski	Sections 7, 11
Runtime and deployment documentation	Mehmet Eski	Sections 15-16
Final review and consistency check	Kerem Elma, Mehmet Eski	Full document